

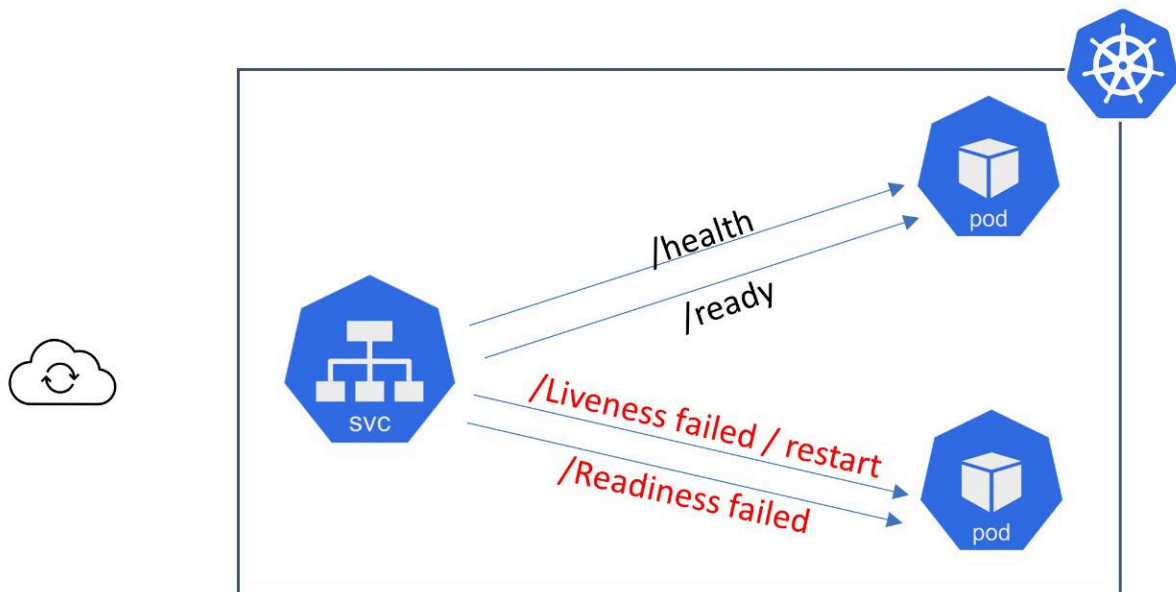
Kubernetes Lab: Health Probes (Liveness, Readiness, Startup)

Concept Overview

In Kubernetes, **health probes** allow the control plane to determine the state of your containers and act accordingly.

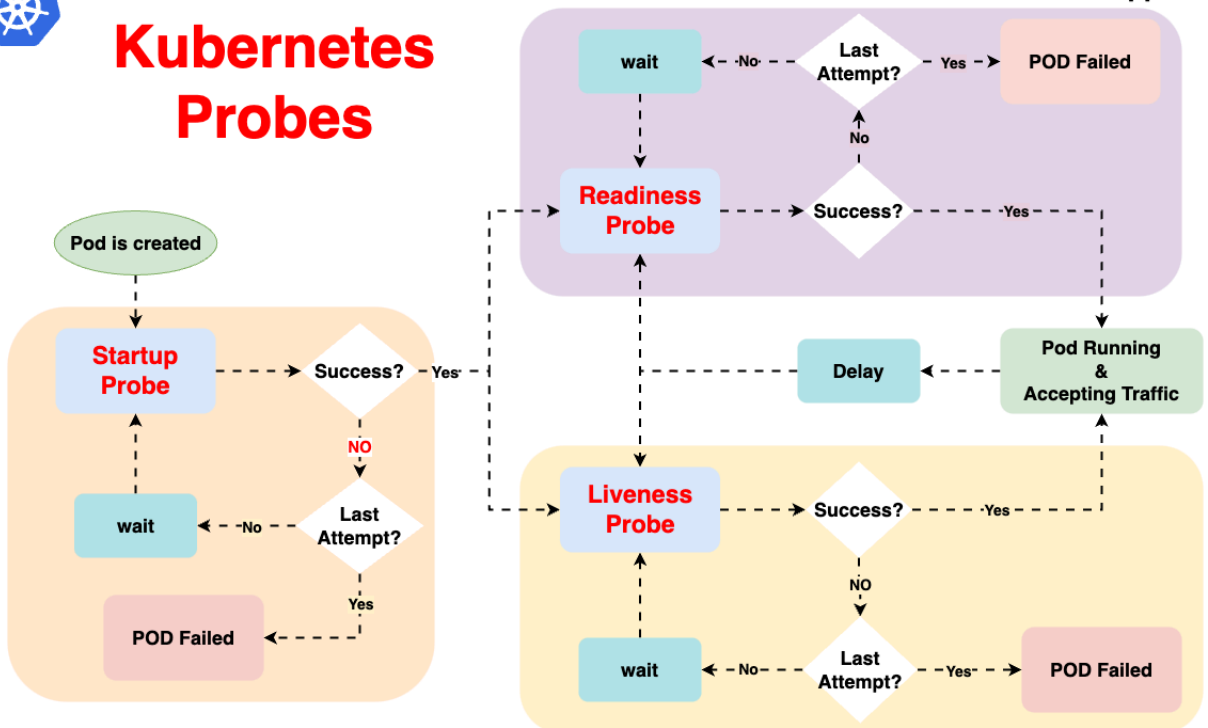
Probe Type	Question It Answers	What Kubernetes Does
Liveness	Is the app still alive?	Restarts the container if it fails
Readiness	Can the app serve traffic?	Removes Pod from Service endpoints
Startup	Has the app finished starting?	Delays liveness/readiness checks

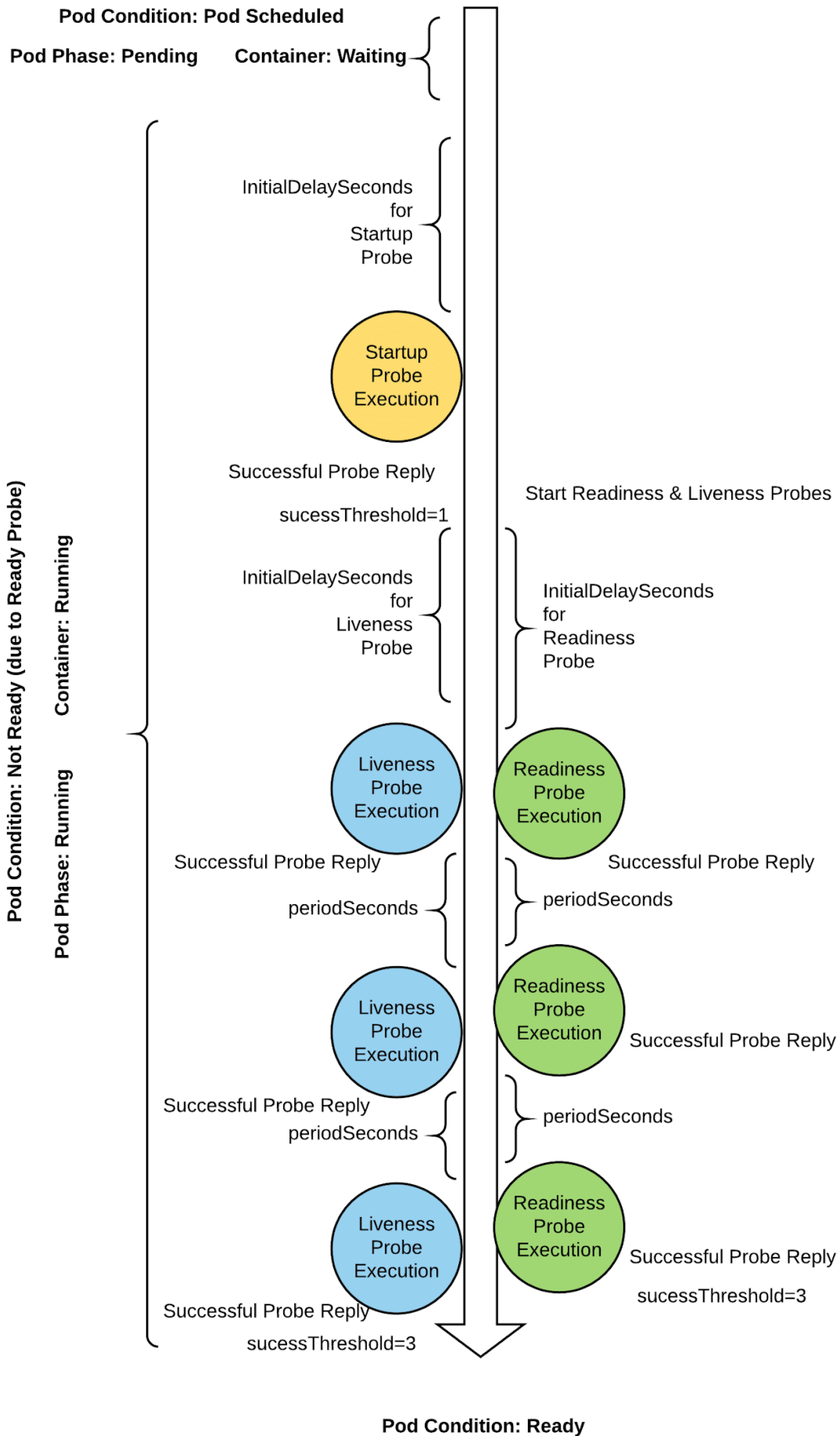
How Probes Work (Mental Model)

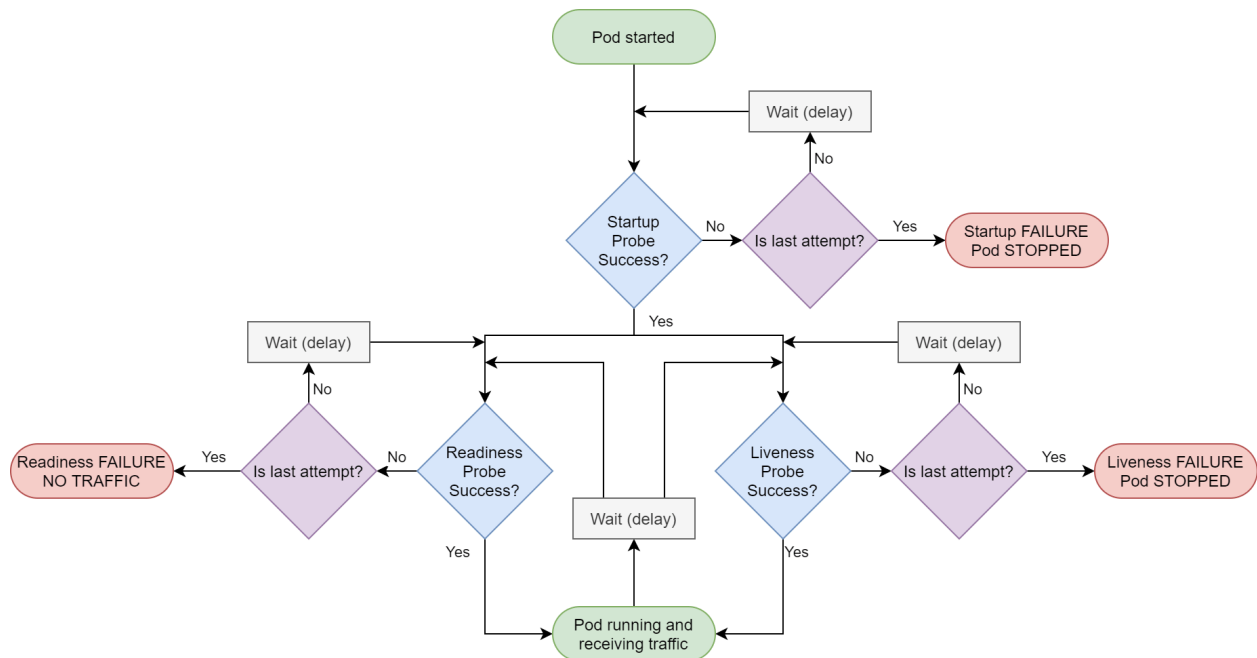


Kubernetes Probes

Anvesh Muppada







Think of probes as Kubernetes asking your container questions:

- **Startup probe** → “Are you done booting?”
- **Liveness probe** → “Are you stuck or broken?”
- **Readiness probe** → “Can I send traffic to you?”

🔧 Lab Part 1: Liveness & Readiness Probes

We'll create a simple Pod using `busybox` to simulate application behavior.

Step 1 Create the YAML File

Create a file named:

health-probe-demo.yaml

Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
  name: health-demo
spec:
  containers:
    - name: web
      image: busybox
      command:
        - sh
        - -c
        - |
          touch /tmp/healthy;
          sleep 20;
          rm -f /tmp/healthy;
          sleep 3600
      livenessProbe:
        exec:
          command:
            - cat
            - /tmp/healthy
          initialDelaySeconds: 5
          periodSeconds: 5
      readinessProbe:
        exec:
          command:
            - cat
            - /tmp/healthy
          initialDelaySeconds: 5
          periodSeconds: 5
```

1. Creates `/tmp/healthy`
2. Waits 20 seconds
3. Deletes `/tmp/healthy`
4. Keeps the container running

Expected Behavior

- First 20 seconds → Pod is healthy
 - After 20 seconds → Liveness probe fails
 - Kubernetes restarts the container
-

Step 2 Apply the Pod

```
kubectl apply -f health-probe-demo.yaml
```

Step 3 Watch It Live

```
kubectl get pod health-demo -w
```

After ~20 seconds, you should see:

- **RESTARTS** count increase
 - Pod cycling due to failed liveness probe
-

Step 4 Inspect Events

```
kubectl describe pod health-demo
```

```
kubectl logs health-demo
```

Look for:

- Liveness probe failed
 - Container restarted
-

Lab Part 2: Startup Probe (For Slow Applications)

Some applications take time to initialize (e.g., loading large datasets).

Without a startup probe:

- Liveness might kill the container too early.
-

Step **1** Create a New File

startup-demo.yaml

apiVersion: v1

kind: Pod

metadata:

 name: startup-demo

spec:

 containers:

 - name: web

 image: busybox

 command:

 - sh

 - -c

 - |

 echo "Starting app...";

 sleep 30;

 touch /tmp/started;

 sleep 3600

 startupProbe:

```
exec:
  command: ["cat", "/tmp/started"]
failureThreshold: 10
periodSeconds: 3
livenessProbe:
  exec:
    command: ["cat", "/tmp/started"]
  initialDelaySeconds: 5
  periodSeconds: 5
readinessProbe:
  exec:
    command:
      - cat
      - /tmp/healthy
  initialDelaySeconds: 5
  periodSeconds: 5
```

What Happens Here?

- App takes **30 seconds** to "start"
- Startup probe checks every 3 seconds
- Liveness probe waits until startup probe succeeds
- Container is NOT restarted during startup

Step Apply It

```
kubectl delete pod health-demo
kubectl apply -f startup-demo.yaml
kubectl get pod startup-demo -w
```


Observe:

- No restarts during the 30-second startup delay
 - Pod becomes ready once `/tmp/started` exists
-

Key Configuration Fields Explained

Field	Meaning
<code>initialDelaySeconds</code>	Wait time before first probe
<code>periodSeconds</code>	How often to run probe
<code>failureThreshold</code>	Failures before marking unhealthy
<code>timeoutSeconds</code>	Probe timeout duration
<code>successThreshold</code>	Successes required to mark healthy

Probe Types

Kubernetes supports three probe mechanisms:

1 Exec Probe

Runs a command inside the container:

exec:

```
command: ["cat", "/tmp/healthy"]
```

HTTP Probe

Sends an HTTP request:

httpGet:

path: /health

port: 8080

TCP Probe

Checks if a TCP port is open:

tcpSocket:

port: 3306



Cleanup

kubectrl delete pod health-demo startup-demo

Reflection Questions (For Students)

1. What happens if you remove the liveness probe?
 2. What happens if readiness fails but liveness succeeds?
 3. Why is startup probe important for slow applications?
 4. How would probes behave in a Deployment instead of a Pod?
-

Bonus Exercise

Modify the first example:

- Increase `failureThreshold`
 - Change `periodSeconds`
 - Observe how restart timing changes
-

Summary

You learned:

- Liveness = restart broken containers
- Readiness = control traffic routing
- Startup = protect slow-starting apps

Health probes are critical for building **self-healing, production-ready workloads** in Kubernetes.
